

Module 2: Inheritance in OOP

Overview

- Overriding
- Shadowing
- Abstract Classes
- Interfaces

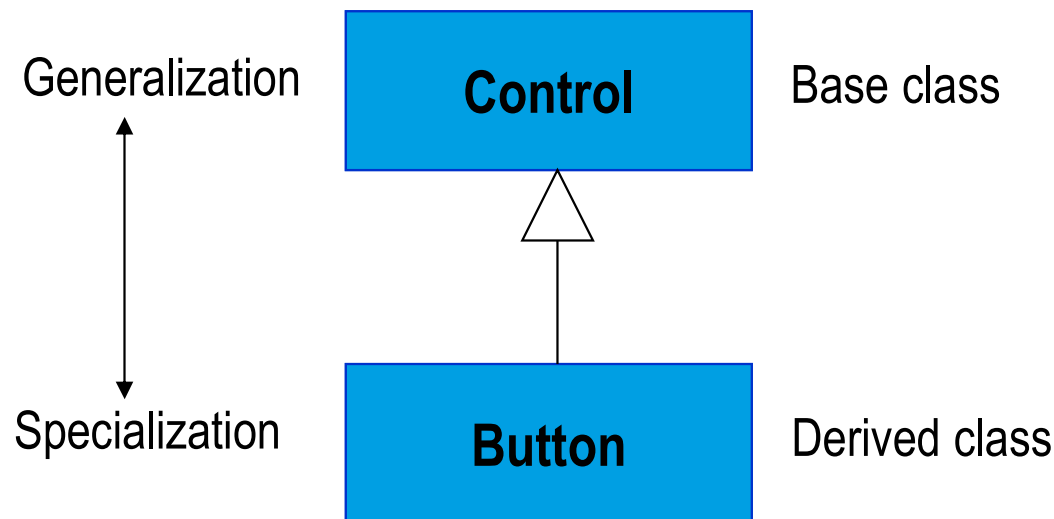
OOP prerequisites and questions

- Value x Reference types
- Class x Object
- Operator this
- Static members
- Using constructors and keyword base



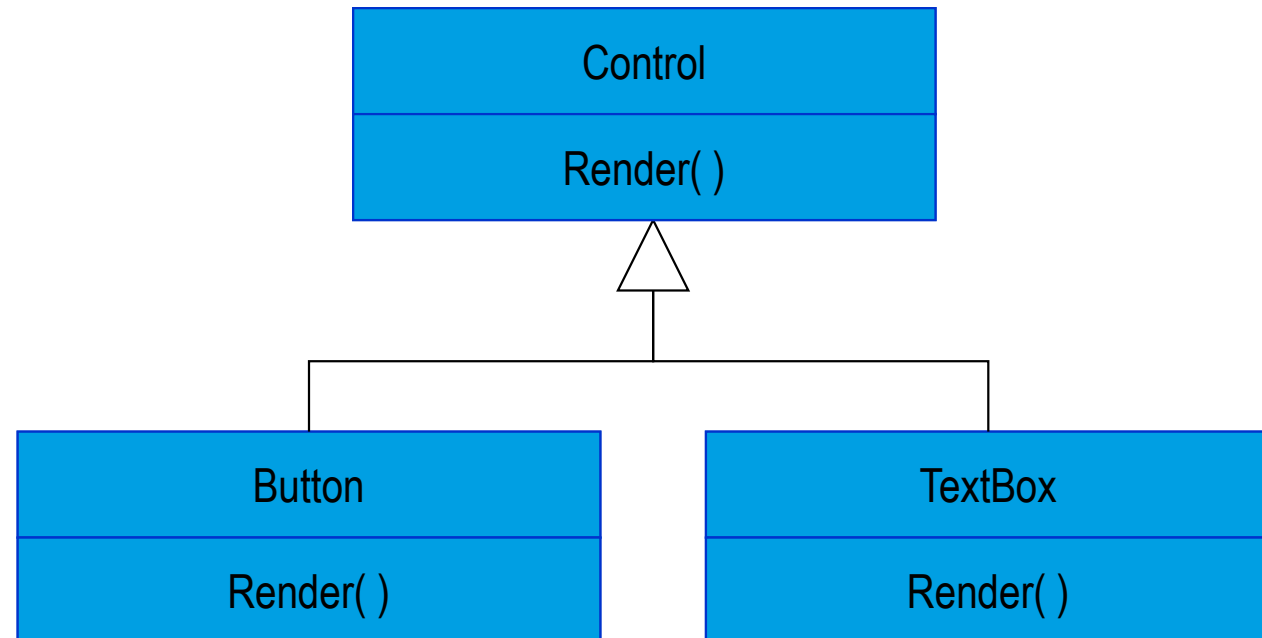
Inheritance

- Inheritance specifies an “is a kind of” relationship
 - Inheritance is a class relationship
 - New classes specialize existing classes



Polymorphism

- Means having many forms
- Implementing Polymorphism in C#
 - Overloading
 - Overriding
 - Shadowing
 - Interfaces



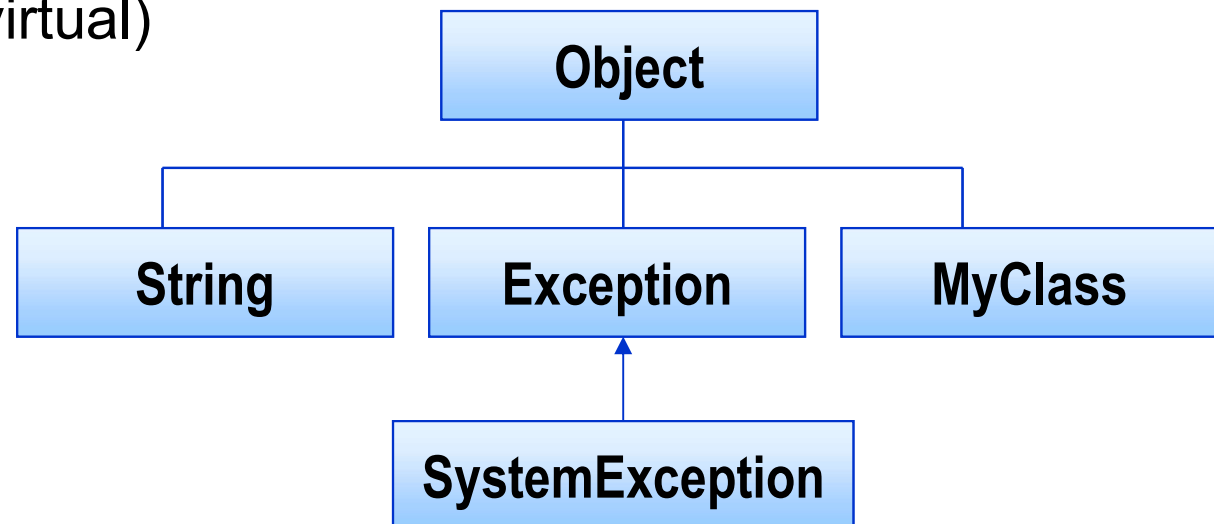
Overriding Methods

- Using override keyword

```
class Token
{
    public virtual string GetName() {}
}
class CommentToken: Token
{
    public override string GetName() { base.GetName();}
}
```

The object Type

- Synonym for System.Object
- Base class for all classes
- Common methods for all reference types
 - **ToString()** (virtual)
 - **Equals()** (virtual)
 - **GetHashCode()** (virtual)
 - **GetType()**



Lab 02.1 - System.Object



Example:

1. Overriding
2. Overriding of Overridden method
3. Overriding ToString()
4. Overriding Equals()
5. Overriding GetHashCode()

Properties vs Public member variables

- Properties provide a level of abstraction allowing you to change the fields while not affecting the external way they are accessed by the things that use your class.
- Property can be read-only
- Property can have different member protection for getter and setter
- Property is implemented as methods, so it can be virtual and it can be overridden
- Property can be accessed by reflection
- There should be no Public data members in your code

So what wrong with properties?

```
private string _firstName;  
  
public string FirstName  
{  
    get { return _firstName; }  
    set { _firstName = value; }  
}
```



Use automatic properties

```
public string FirstName { get; set; }
```

- the compiler can automate creating the private field and the default get/set operations for you

```
private string <FirstName>k__BackingField;  
  
public string FirstName  
{  
    get  
    {  
        return this.<FirstName>k__BackingField;  
    }  
    set  
    {  
        this.<FirstName>k__BackingField = value;  
    }  
}
```



Private setter as readonly property

```
public string FirstName { get; set; }
```

- ReadOnly (set only in ctor)

```
public string FirstName { get; }
```

- ReadOnly with initialization

```
public string Color { get; } = "Blue";
```

- ReadOnly with private setter

```
public string FirstName { get; private set; }
```

Lab 02.2 - Automatic properties



Exercise:

1. Classic property
2. Automatic properties
3. Overriding properties
4. Indexer
5. Overloading Indexer

Shadowing (Member Hiding)

- Using new keyword to Hide Methods

```
class Token
{
    ...
    public int GetLineNumber() { ... }
}
class CommentToken: Token
{
    ...
    new public int GetLineNumber() { ... }
}
```

Overriding vs Shadowing

- Using Overriding

```
class Token
{
    ...
    public virtual string GetName( ) { ... }
}
class CommentToken: Token
{
    ...
    public override string GetName( ) { base.GetName();}
}
```

- Using Shadowing

```
class Token
{
    ...
    public int GetLineNumber() { ... }
}
class CommentToken: Token
{
    ...
    new public int GetLineNumber() { ... }
}
```

Lab 02.3 – Overriding and Shadowing



Example:

1. Overriding
2. Shadowing
3. Shadowing trick

Using Sealed Classes

- A sealed class cannot be inherited
 - cannot be used as a base class
- Methods are no longer virtual
 - run-time optimizations can make calling sealed class members slightly faster
- String and StringBuilder are sealed

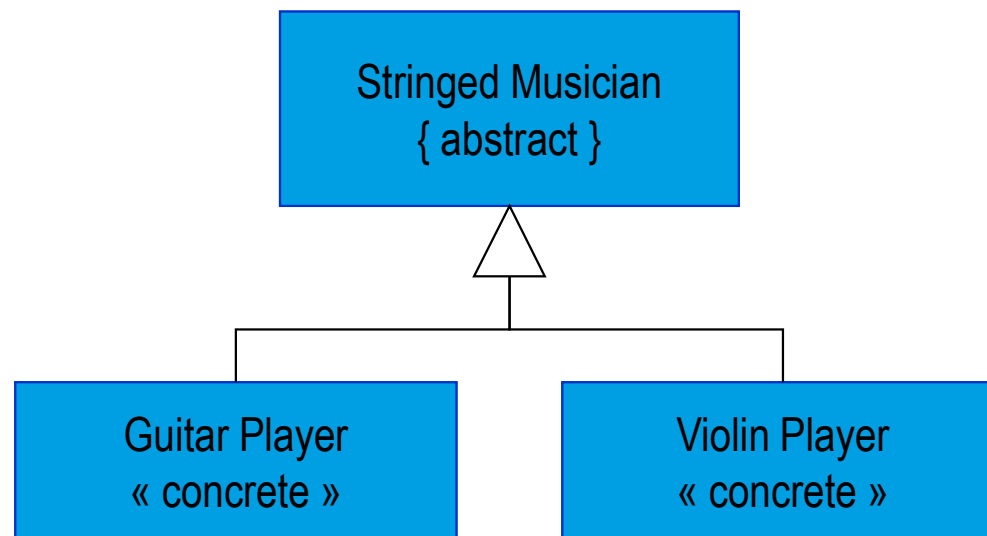
```
namespace System
{
    public sealed class String
    {
    }
}
namespace MyNamespace
{
    class FancyString: String { ... }
}
```



FAIL

Abstract Base Classes

- You cannot create instances of abstract classes
- Abstract class can contain abstract method



Declaring Abstract Classes

- Use the abstract keyword

```
abstract class Token
{
    ...
}
class Test
{
    static void Main()
    {
        new Token();
    }
}
```

FAIL

Implementing Abstract Methods

- Use the abstract keyword
- Abstract methods are virtual

```
abstract class Token
{
    public virtual string GetName() { ... }
    public abstract int GetLength();
}
class CommentToken : Token
{
    public override string GetName() { ... } // can override
    public override int GetLength() { ... } // must override
}
```

- Only abstract classes can declare abstract methods
- Abstract methods cannot contain a method body

Lab 02.4 - Abstract class



Example:

1. Defining abstract class
2. Generalization and abstract class

Using Interfaces

- Contains only the signatures of methods, properties, events or indexers
- You cannot create instances of an interface.
- Interface can inherit from one or more base interfaces.
- A class that implements an interface
 - **Explicit implementation**
member accessed only through an interface
 - **Implicit implementation**
member accessed through an interface or directly from a class instance

Declaring Interfaces

- Interface is declaration of
 - Methods
 - Properties
 - Indexers
 - Events

```
interface IToken
{
    int GetLineNumber();
    void SetName(string name);
}
```

Explicit implementation of Interface

- Use the fully qualified interface method name

```
interface IToken
{
    int GetLineNumber();
    void SetName(string name);
}
class Token : IToken
{
    int IToken.GetLineNumber()
    {
    }
    void IToken.SetName(string name)
    {
    }
}
```


Implicit implementation of Interface

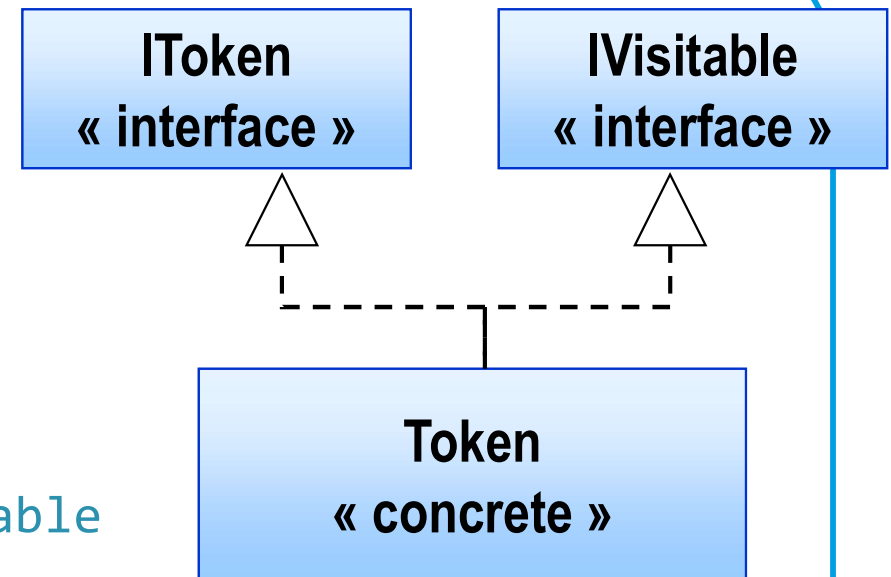
- The signature of implementing method must be the same as the interface method
- The implementing method can be virtual or non-virtual

```
interface IToken
{
    int GetLineNumber();
    void SetName(string name);
}
class Token : IToken
{
    public virtual int GetLineNumber()
    {
    }
    public void SetName(string name)
    {
    }
}
```

Implementing Multiple Interfaces

- A class can implement zero or more interfaces

```
interface IToken
{
    string GetName( );
}
interface IVisitable
{
    void Accept(IVisitor v);
}
class Token: Object, IToken, IVisitable
{
}
```



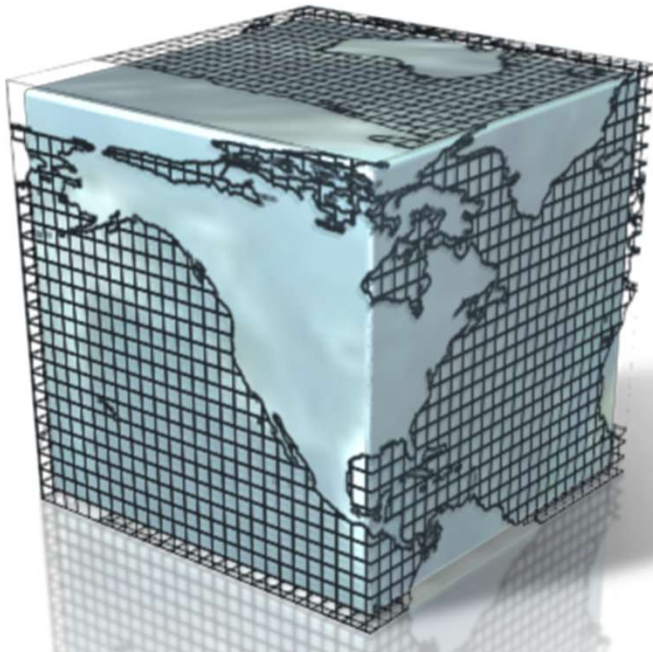
Lab 02.5 - Interface



Example:

1. Create new Interface IPayment
2. Explicit and Implicit implementation
3. Using Payment manager
4. Extending application

Exercise - Interface



- Create an interface **IVehicle** with two methods:
 1. **Start()** – starts the vehicle
 2. **Stop()** – stops the vehicle
- Then, implement a class **Car** that:
 1. **Implicitly** implements the **Start()** method.
 2. **Explicitly** implements the **Stop()** method (meaning it can only be called through an **IVehicle** reference)



Finally, introduce class **Garage**, which test the vehicle for Start and Stop using Method **TestVehicle()**.

Expected output:

```
Car is starting...  
Car is stopping...
```

Review

- Overriding
- Shadowing
- Abstract Classes
- Interfaces